

## PREDICTING OCCUPANCY EFFICIENCY IN SEATTLE AIRBNB LISTINGS

Vishesha Bitla, Keerthi Sri Cherukuri, Gafaruddin Shaik, Jhansimala Silla

Missouri University of Science and Technology, Rolla, USA

[vbgfn@mst.edu](mailto:vbgfn@mst.edu); [kc6bd@mst.edu](mailto:kc6bd@mst.edu); [gs9cb@mst.edu](mailto:gs9cb@mst.edu); [js6p6@mst.edu](mailto:js6p6@mst.edu)

**Abstract** - This project investigates Airbnb listings in Seattle to identify key factors influencing short-term occupancy efficiency and to predict whether a listing will achieve Very High Occupancy within a 30-day period. Unlike revenue-oriented research, this work prioritizes occupancy as a more actionable measure for hosts. The dataset underwent cleaning, feature engineering, and variance reduction through VIF analysis. Multiple machine-learning models were implemented, including Logistic Regression, Random Forest, Naive Bayes, and Decision Tree. Results were evaluated using accuracy, precision, recall, and F1-score. The Decision Tree model demonstrated the strongest balance between recall and precision while offering high interpretability and actionable insights. Findings indicate that pricing strategy, review volume, and revenue bands strongly influence booking probability.

### Introduction

Vacancy directly impacts host earnings in short-term rental markets. Predicting occupancy helps owners modify pricing, listing quality, and availability in near-real-time. A 30-day prediction window enables tactical adjustments rather than long-term reactive strategies.

The goal of this project is to identify factors that influence occupancy and evaluate machine-learning models that classify listings based on whether they fall under high or low occupancy classes

### Research Question

Which variables most strongly influence the 30-day occupancy of Airbnb listings, and which algorithm best predicts high occupancy outcomes?

### Objective

To prepare data, engineer features, implement models, evaluate performance, and derive actionable strategies for Airbnb hosts.

### Background

Most Airbnb analytics research focuses on price or yearly revenue forecasting. However, high occupancy is a prerequisite for high revenue. To bridge this gap, this study forms an occupancy-centered approach. Predicting short-term occupancy equips hosts with insights to increase bookings, optimize pricing, and improve guest experience.

### Dataset Description

The dataset was sourced from Inside Airbnb (Seattle, WA). The original listing file contained 6862 rows and 80 features. After filtering irrelevant/missing entries, the final dataset used for modeling consisted of 4469 listings with 18 selected variables.

### Engineered Variables

Two new columns were created to align with a 30-day occupancy objective:

$\text{estimated\_occupancy\_30} = 30 - \text{availability\_30}$

$$\text{estimated\_revenue\_30} = \text{price} \times \text{estimated\_occupancy\_30}$$

These variables represent short-term demand and revenue potential more accurately than annual metrics.

### Data Cleaning and Preprocessing

Price and percentage fields were converted to numeric format after removing symbols. Missing values were handled based on contextual relevance. Extreme pricing outliers were capped using percentile thresholds. Occupancy rate was computed as:

$$\text{occupancy\_rate\_30} = \frac{\text{estimated\_occupancy\_30}}{30}$$

To prevent multicollinearity in modelling, VIF-based feature elimination was performed until variance inflation values were within acceptable ranges.

The cleaned dataset became suitable for supervised learning with consistent formats and reduced noise.

### Exploratory Data Analysis

Exploratory plots revealed right-skewed pricing and availability distributions. Listings with more reviews tended to have higher occupancy, while high availability implied lower booking frequency. Revenue and occupancy displayed non-linear relationships, suggesting suitability for tree-based models.

**Chart 1 – Price Distribution Histogram**

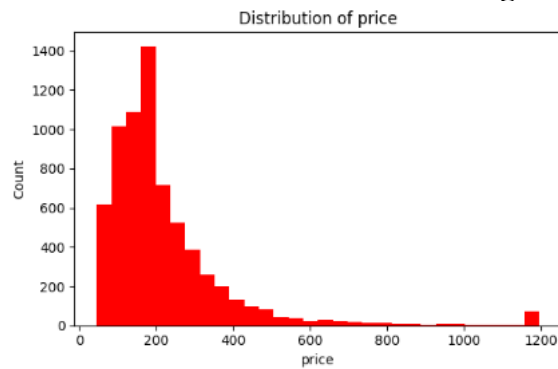


Figure1. Distribution of listing prices showing right-skew towards high-priced properties.

**Chart 2 – Availability 30 Distribution Chart**

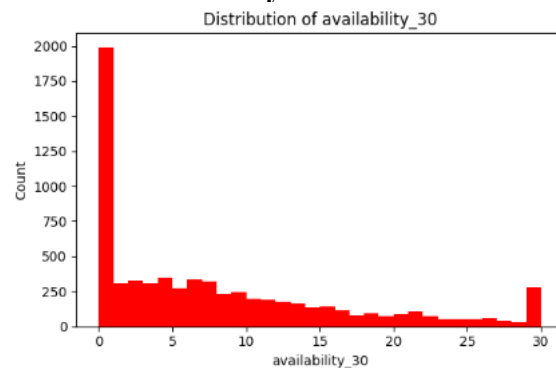


Figure 2. Distribution of availability days in a 30-day window.

**Chart 3 – Correlation Heatmap**

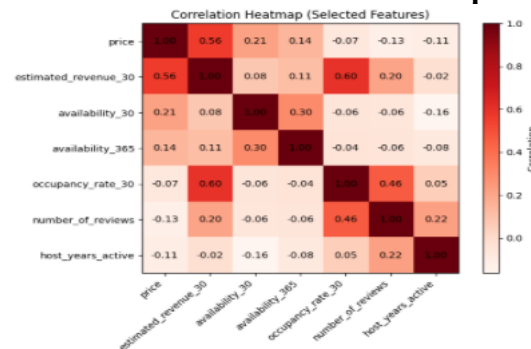


Figure 3. Correlation heatmap showing relationship between numeric features.

**Chart 4 – Number of Reviews vs Occupancy Plot**

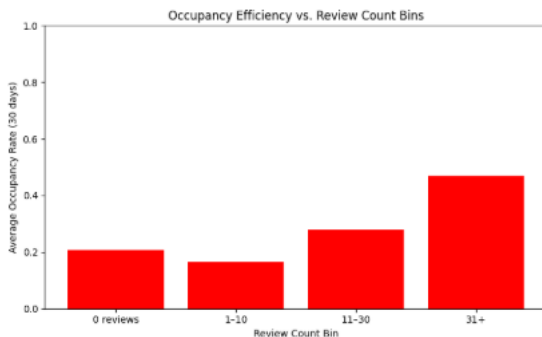


Figure 4. Review count impact showing higher occupancy in listings with review volume.

### Target Transformation & Quartile Logic

The continuous target variable `occupancy_rate_30` was transformed into a binary class variable using quartile segmentation. Quartiles (Q1–Q4) divide a distribution into four equal frequency groups. In our dataset, listings falling in the top quartile (Q4), representing the highest 25% of occupancy rates, were labeled as Very High Occupancy (1). Listings in Q1 through Q3 (bottom 75%) were grouped as Not Very High Occupancy (0). This segmentation ensures that the model focuses on identifying truly high-performing listings rather than moderately booked ones, creating clearer separation for classification.

| Quartile | Meaning                    | Portion of Data |
|----------|----------------------------|-----------------|
| Q1       | Lowest 25% occupancy       | Bottom 0-25%    |
| Q2       | Next 25%                   | 25-50%          |
| Q3       | Next 25%                   | 50-75%          |
| Q4       | Highest occupancy listings | Top 75-100%     |

### Final Classification Rule

- Top quartile (Q4) - Very High Occupancy = 1
- Q1–Q3 (bottom 75%) - Not Very High Occupancy = 0

### Performance Interpretation of All Models

**Linear Regression** served as a baseline model to observe linear relationships between features and occupancy rate. While it provides general trend understanding; it was unsuitable for classification because occupancy efficiency is categorical in this application. As expected, it underperformed relative to classification-focused models.

### Linear Regression Results

| Train r2 | Test r2 | Test MAE | Test RMSE |
|----------|---------|----------|-----------|
| 0.6795   | 0.6927  | 0.1088   | 0.1475    |

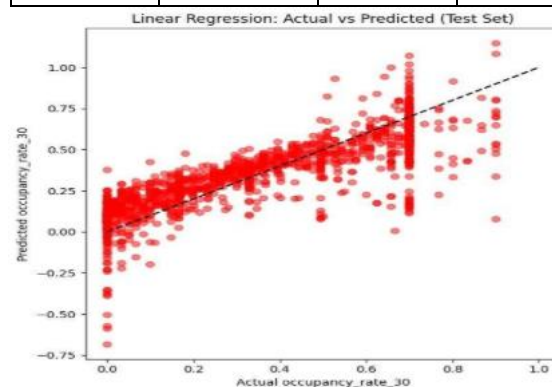


Figure 5. Linear regression Actual vs predicted.

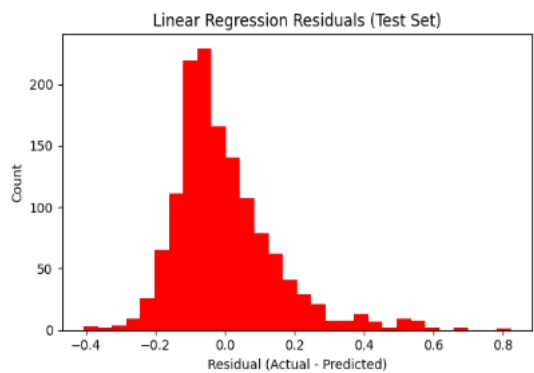


Figure 6. Linear Regression Residuals.

**Logistic Regression** mapped features occupancy probability and allowed threshold tuning between 0.1–0.9. At lower threshold values, recall was high, but precision dropped, while stricter thresholds produced the reverse effect. To evaluate the model fairly, we selected a cutoff value of 0.4, which offered the most balanced trade-off between precision and recall during threshold comparison.

**Logistic Regression Confusion Matrix at cutoff =0.4**

| True Positive | False Positive | True Negative | False Negative |
|---------------|----------------|---------------|----------------|
| 287           | 60             | 978           | 48             |

**Logistic Regression Performance Metrics at cutoff=0.4**

| Accuracy | Precision | Recall | F1-Score |
|----------|-----------|--------|----------|
| 0.9213   | 0.8271    | 0.8567 | 0.8416   |

Threshold tuning revealed a measurable trade-off between recall and Precision. At thresholds close to 0.1, recall remained very high (detecting most Very High listings) but precision decreased due to false positives. Toward 0.7–0.9, precision increased while recall sharply dropped, indicating stricter classification behavior. The mid-range threshold approximately 0.4 (selected cutoff) achieved the most stable F1 score and provided a reasonable balance, though still less flexible compared to non-linear tree-based models.

**Cutoff comparison table**

| Threshold  | TP         | FP        | TN         | FN        |
|------------|------------|-----------|------------|-----------|
| 0.1        | 303        | 198       | 840        | 32        |
| 0.2        | 301        | 132       | 906        | 34        |
| 0.3        | 293        | 85        | 953        | 42        |
| <b>0.4</b> | <b>287</b> | <b>60</b> | <b>978</b> | <b>48</b> |
| 0.5        | 269        | 46        | 992        | 66        |
| 0.6        | 252        | 25        | 1013       | 83        |
| 0.7        | 231        | 14        | 1024       | 104       |
| 0.8        | 189        | 3         | 1035       | 146       |
| 0.9        | 134        | 0         | 1038       | 201       |

| Threshold  | Recall        | Specificity   | J_Stat        |
|------------|---------------|---------------|---------------|
| 0.1        | 0.9045        | 0.8092        | 0.7137        |
| 0.2        | 0.8985        | 0.8728        | 0.7713        |
| 0.3        | 0.8746        | 0.9181        | 0.7927        |
| <b>0.4</b> | <b>0.8567</b> | <b>0.9422</b> | <b>0.7989</b> |
| 0.5        | 0.8030        | 0.9557        | 0.7587        |
| 0.6        | 0.7522        | 0.9759        | 0.7282        |
| 0.7        | 0.6896        | 0.9865        | 0.6761        |
| 0.8        | 0.5642        | 0.9971        | 0.5613        |
| 0.9        | 0.4000        | 1.0000        | 0.4000        |

**Random Forest** performed strongly and captured non-linear interactions. Its multiple-tree ensemble structure increased predictive power and reduced variance;

however, interpretability declined relative to the Decision Tree. Although accuracy was competitive, the lack of transparent decision rules made it less practical for explaining results to hosts.

#### Random Forest Confusion Matrix

| True Positive | False Positive | True Negative | False Negative |
|---------------|----------------|---------------|----------------|
| 301           | 47             | 991           | 34             |

#### Random Forest Performance Metrics

| Accuracy | Precision | Recall | F1-Score |
|----------|-----------|--------|----------|
| 0.9410   | 0.8649    | 0.8985 | 0.8814   |

**Naive Bayes** offered fast computation and simplicity, but its independence assumption among features limited performance in real rental data where interactions exist (price x reviews, revenue  $\times$  availability). This model served mainly as a benchmark reference.

#### Naive Bayes Confusion Matrix

| True Positive | False Positive |  | True Negative | False Negative |
|---------------|----------------|--|---------------|----------------|
| 315           | 803            |  | 235           | 20             |

#### Naive Bayes Performance Metrics

| Accuracy | Precision | Recall | F1-Score |
|----------|-----------|--------|----------|
| 0.4006   | 0.2818    | 0.9403 | 0.4336   |

**Decision Tree** in contrast, balanced interpretability and performance most effectively. The model achieved the highest precision-recall balance among all classifiers and provided human-readable rules. This made it the most practical for explaining actionable factors affecting occupancy, ultimately leading to its selection as the final model for the study.

#### Decision Tree Confusion Matrix

| True Positive | False Positive | True Negative | False Negative |
|---------------|----------------|---------------|----------------|
| 302           | 39             | 999           | 33             |

#### Decision Tree Performance Metrics

| Accuracy | Precision | Recall | F1-Score |
|----------|-----------|--------|----------|
| 0.9476   | 0.8856    | 0.9015 | 0.8935   |

#### Model Evaluation Results

| Model               | TP  | FP  | TN  | FN |
|---------------------|-----|-----|-----|----|
| Logistic Regression | 287 | 60  | 978 | 48 |
| Random Forest       | 301 | 47  | 991 | 34 |
| Decision Tree       | 302 | 39  | 999 | 33 |
| Naive Bayes         | 315 | 803 | 235 | 20 |

| Model               | Accuracy | Precision | Recall | F1     |
|---------------------|----------|-----------|--------|--------|
| Logistic Regression | 0.9213   | 0.8271    | 0.8567 | 0.8416 |
| Random Forest       | 0.9410   | 0.8649    | 0.8985 | 0.8814 |
| Decision Tree       | 0.9476   | 0.8856    | 0.9015 | 0.8935 |
| Naive Bayes         | 0.4006   | 0.2818    | 0.9403 | 0.4336 |



Figure 7. Model Performance.

### Decision Tree Interpretation

The final model chose `log_estimated_revenue_30` as the primary node. Listings below this revenue signal were mostly predicted as Not Very High Occupancy. For the higher-revenue group, `price`  $\leq 147.5$  significantly increased the probability of Very High Occupancy.

High occupancy probability was most common in listings with:

- Moderate pricing
- Higher revenue bands
- Strong review volume

Premium pricing at similar revenue levels reduced the likelihood, suggesting price sensitivity.



Figure 8. Decision tree

[http://decision\\_tree\\_flow\\_diagram](http://decision_tree_flow_diagram)

### Review Features & Their Effect on Occupancy

To evaluate whether guest feedback contributes to booking frequency, we merged the review dataset with the listing dataset using the `listing_id` field. Review-related attributes such as `number_of_reviews`, `review_scores_rating`, and `review_recency` were evaluated against occupancy rate distributions.

### Key Observations:

1. Listings with more reviews tended to achieve higher occupancy, indicating social proof influences guest willingness to book.
2. `review_scores_rating` showed moderate correlation, meaning even highly rated listings do not perform well without sufficient review volume.
3. Review count acted as a trust-building factor, especially for new guests browsing competitive listings.
4. Listings with fewer than 10 reviews showed inconsistent occupancy, suggesting that early review of accumulation is critical to initial traction.

| Review Group   | Average Occupancy Rate Trend |
|----------------|------------------------------|
| 0–10 reviews   | Low & unstable occupancy     |
| 11–50 reviews  | Noticeable improvement       |
| 50–200 reviews | Strong occupancy trend       |
| 200+ reviews   | Highest booking consistency  |

*Finding: Reviews do not just reflect guest satisfaction; they actively influence demand visibility and conversion probability.*

### Feature Importance & SHAP Interpretation

To understand how each feature contributed to occupancy classification, SHAP values (Shapley Additive explanations) were computed on the best-performing Decision Tree model. SHAP assigns contribution scores to features, allowing us to interpret prediction behavior beyond raw accuracy metrics.

Insights from SHAP Analysis:

1. `log_estimated_revenue_30` & `estimated_revenue_30` had the strongest positive influence.

2. Price contributed nonlinearly; moderate pricing improved SHAP scores while very high pricing reduced them.
3. Number\_of\_reviews ranked as a significant positive feature, supporting earlier findings that social validation increases demand.
4. availability\_30 and availability\_365 have negative SHAP influence higher availability generally implies lower bookings.

## Appendix

The appendix below contains only the key sections of code and major output elements relevant to understanding preprocessing, feature engineering, modelling, and evaluation results.

### Appendix A – Data Cleaning & Feature Engineering Code Snippets

```
# Inputs: 30-day Occupancy & Revenue
if ['estimated_occupancy_30', 'availability_30'].issubset(df.columns):
    mask_occ = df['estimated_occupancy_30'] >= 20 & df['availability_30'] > 0
    df.loc[mask_occ, 'estimated_occupancy_30'] = df.loc[mask_occ, 'availability_30']
    print("Imputed estimated_occupancy_30 for [mask_occ.sum()] rows using availability_30")

if ['estimated_revenue_30', 'price'].issubset(df.columns):
    mask_rev = (
        (df['estimated_revenue_30'] >= 0) &
        (df['estimated_occupancy_30'] >= 10) &
        (df['price'] > 0)
    )
    df.loc[mask_rev, 'estimated_revenue_30'] = (
        df.loc[mask_rev, 'estimated_occupancy_30'] * df.loc[mask_rev, 'price']
    )
    print(f"Imputed estimated_revenue_30 for [mask_rev.sum()] rows using price * occupancy_30")
df['estimated_revenue_30'] = df['estimated_revenue_30'].clip(lower=0)
```

Figure 9. 30 days of occupancy and revenue.

### Appendix B – Model Training Code Highlights

```
try:
    y_bins_reg = pd.qcut(y_cont, q=5, labels=False, duplicates='drop')
except ValueError:
    y_bins_reg = pd.qcut(y_cont.rank(method="first"), q=5, labels=False)

X_train, X_test, y_train, y_test = train_test_split(
    X_all, y_cont, test_size=0.2, stratify=y_bins_reg, random_state=42
)

num_cols_reg = X_train.select_dtypes(include="number").columns.tolist()
preprocess_reg = ColumnTransformer(
    transformers=[('num', StandardScaler(), num_cols_reg)],
    remainder="passthrough"
)
```

Figure 10. Model training

### Appendix C – Evaluation Output

| Model Comparison Summary |                     |        |     |     |     |    |          |           |
|--------------------------|---------------------|--------|-----|-----|-----|----|----------|-----------|
|                          | Model               | Cutoff | TP  | FP  | TN  | FN | Accuracy | Precision |
| 0                        | Logistic Regression | 0.4    | 287 | 68  | 978 | 48 | 0.9213   | 0.8271    |
| 1                        | Random Forest       | 0.4    | 301 | 47  | 991 | 34 | 0.9410   | 0.8649    |
| 2                        | Decision Tree       | 0.4    | 302 | 39  | 999 | 33 | 0.9476   | 0.8856    |
| 3                        | Naive Bayes         | 0.4    | 315 | 803 | 235 | 20 | 0.4006   | 0.2818    |

|   | Recall | F1     |
|---|--------|--------|
| 0 | 0.8567 | 0.8416 |
| 1 | 0.8985 | 0.8814 |
| 2 | 0.9015 | 0.8935 |
| 3 | 0.9483 | 0.4336 |

Figure 11. Model summary table

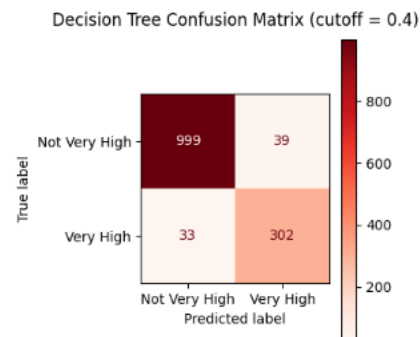


Figure 12. Decision tree confusion matrix

## Discussion

The models collectively demonstrate that occupancy is driven by revenue behavior, pricing decisions, review leverage, and listing engagement. Decision Tree, Random Forest, and Logistic Regression align in identifying revenue-based attributes as top predictors. The incorporation of review features highlighted the importance of guest validation in supporting sustained occupancy.

The threshold comparison analysis for Logistic Regression indicated that a 0.4 cutoff balanced recall and precision most effectively for this dataset.



However, tree-based models captured non-linear relationships more robustly, leading to superior performance. Decision Tree's interpretability stands out as well-suited for hosts who require immediate guidance without a deep technical background.

By translating model nodes into decision logic, property owners can optimize price, encourage reviews, and monitor availability behavior in real time.

### Conclusion

This research presents a machine-learning approach for predicting 30-day occupancy efficiency of Seattle Airbnb listings. After data cleaning, feature engineering, and model comparison, the Decision Tree model demonstrated the best balance of predictive performance and interpretability.

The findings highlight three actionable levers for hosts:

1. Maintain optimal pricing, neither too high nor too low.
2. Actively collect guest reviews to build credibility.
3. Optimize booking availability to avoid under/over-exposure

Future work may extend this model with seasonal trend integration, dynamic pricing, sentiment analysis on review text, and SHAP-based personalized recommendations

### References

- Wang, D., & Nicolau, J. L. (2017). Price determinants of sharing economy accommodation: A study of listings from 33 cities on Airbnb.com. *International Journal of Hospitality Management*, 62, 120–131.  
<https://doi.org/10.1016/j.ijhm.2016.12.007>
- Zhang, Z., & Wei, L. (2019). Predicting short-term rental prices using machine learning: A case study of Airbnb in San Francisco. *Journal of Information and Data Management*, 10(3), 200–214.